



RISC-V RV32I Processor: From Single-Cycle to a 5-Stage Pipelined Architecture

The domain of the Project:
Integrated VLSI Design

Team Mentors (and their designation):

Razi Ahmed & DeviPrasad Mishra

Team Members:
Mr. Vishal Gowda V N

Period of the project

June 2026 to July 2026



DECLARATION

The project titled “**RISC-V RV32I Processor: From Single-Cycle to a 5-Stage Pipelined Architecture**” has been mentored by **Razi Ahmed & DeviPrasad Mishra**, organised by SURE Trust, from April 2023 to August 2023, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Member :

Mr. Vishal Gowda V N

Signature

Mentor's Name : Razi Ahemad

Designation—

Company Name

Mentor's Name : Deviprasad Mishra

Designation—

Company Name

Prof. Radhakumari

Executive Director & Founder

SURE Trust



Table of contents

1. Executive summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry relevance of the project
6. Learning & Reflection
7. Future Scope & Conclusion



EXECUTIVE SUMMARY

This project presents the design, implementation, and functional verification of a 32-bit RISC-V processor supporting the RV32I base integer instruction set, built around a classic five-stage pipeline — Instruction Fetch, Decode, Execute, Memory Access, and Write Back. The processor was described entirely in Verilog HDL and verified through simulation in Icarus Verilog, with waveform analysis in GTK Wave.

The motivation was to gain a working, register-transfer-level understanding of how a modern pipelined processor is built — not just at the level of a textbook block diagram, but with every control signal, pipeline register, and hazard condition accounted for explicitly in code. RV32I was chosen as it is the base integer subset of the open RISC-V architecture: compact enough for a final-year project, yet representative of a real, production-grade ISA.

The final design implements all 37 targeted RV32I instructions across R-type, I-type, load, store, branch, jump, and upper-immediate categories. A dedicated Forwarding Unit resolves most data hazards without stalling, and a Hazard Detection Unit inserts a single-cycle bubble for the load-use case that forwarding cannot resolve. Branch and jump outcomes are resolved in the Execute stage, with the pipeline flushed on any taken branch or jump. All modules were individually written, integrated through a top-level module, and verified against a self-checking testbench, confirming correct arithmetic, memory, branch, and hazard-resolution behaviour across a range of hand-written test programs. The project reinforced core concepts in computer organisation and hardware verification, and lays a solid foundation for extensions such as branch prediction, caching, and the RV32M multiply/divide extension.



Introduction

1.1 Background and Motivation

Computer architecture courses typically begin with the single-cycle processor, where every instruction completes in one clock cycle. This model is easy to understand but inefficient, since the clock period must accommodate the slowest instruction. Pipelining addresses this by splitting execution into stages that operate concurrently on different instructions, much like an assembly line — the architectural technique underlying virtually all modern general-purpose processors. This project was undertaken to move beyond the single-cycle model and build a genuinely pipelined processor from first principles.

RISC-V was chosen as the target ISA because it is open, free of licensing restrictions, and simple enough for an academic implementation while remaining a real, actively used architecture in industry and research. The RV32I base integer subset provides a compact but complete set of arithmetic, logical, memory, and control-flow instructions — large enough to run meaningful programs, yet tractable within the scope of a final-year project.

1.2 Problem Statement, Scope, and Innovation

The goal was to design, in Verilog HDL, a 32-bit processor that correctly implements RV32I using a five-stage pipeline, and to verify — through simulation rather than assumption — that data and control hazards are handled correctly. A pipeline that executes instructions correctly in isolation but produces wrong results when instructions interact through shared registers, memory, or branches is not a functioning pipeline; hazard handling was treated as a first-class requirement rather than an afterthought.

The scope is limited to the RV32I base integer set: extensions such as RV32M (multiply/divide), RV32F/D (floating point), and privileged/CSR instructions were not implemented, and the design excludes caching, branch prediction, and exception/interrupt handling — branches are resolved with a simple not-taken/flush approach. Instruction and data memories are modelled behaviourally for simulation rather than as synthesisable SRAM, and the design was verified functionally in simulation rather than taken through synthesis or FPGA deployment. These boundaries let the effort concentrate on getting the pipeline datapath and hazard-handling logic — the architectural core of the project — correct.



While the five-stage pipeline itself is a well-established architecture, the innovation here lies in the disciplined, from-scratch implementation of every hazard-handling mechanism — forwarding paths from both EX/MEM and MEM/WB, a hazard detection unit that distinguishes a genuine load-use hazard from a forwardable dependency, and explicit flush logic driven by branch/jump resolution — rather than relying on a pre-built IP core. Each control signal (PC_Write, IF_ID_Write, Control_Zero, Flush_D, Flush_E) was reasoned about individually and verified in waveform.

2. Project Objectives

2.1 Defined Objectives

- Design a 32-bit datapath capable of executing the RV32I base integer instruction set.
- Implement the datapath as a classic five-stage pipeline (IF, ID, EX, MEM, WB) in Verilog HDL.
- Design the four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) that isolate stages and hold instruction state between cycles.
- Implement a Forwarding Unit that resolves data hazards by routing results from later stages back to Execute, without unnecessary stalling.
- Implement a Hazard Detection Unit that identifies load-use hazards and stalls the pipeline by exactly one cycle when forwarding cannot resolve the dependency.
- Implement correct branch and jump handling, including flushing incorrectly fetched instructions when a branch is taken.
- Verify the complete design through simulation in Icarus Verilog and waveform analysis in GTKWave.

2.2 Expected Outcomes and Deliverables

- A complete, synthesisable-style Verilog RTL implementation of the RV32I five-stage pipeline, organised into clearly separated modules.
- A self-checking testbench capable of loading test programs into instruction memory and exercising the processor.



- GTKWave simulation waveforms demonstrating correct execution of R-type, I-type, load, store, and branch instruction sequences.
- A working demonstration of hazard resolution: at least one data hazard resolved purely by forwarding, and one load-use hazard resolved by a pipeline stall.
- A final report documenting architecture, module behaviour, verification methodology, and results.

3. Methodology & Results

3.1 Development Approach

The project was developed incrementally rather than attempting the entire pipeline in a single pass. The datapath was first sketched as a block diagram showing the five stages and the signals crossing each pipeline register. Individual combinational and sequential modules — program counter, instruction memory, register file, immediate generator, ALU, and data memory — were written and independently tested with small, targeted testbenches before integration. Once each module behaved correctly in isolation, the four pipeline registers were introduced to connect the stages, and the design was re-tested under clocked operation.

After the base pipeline was functionally correct for hazard-free sequences, hazard-handling logic was added in two further passes: first the Forwarding Unit, to resolve the majority of read-after-write hazards without stalling, and then the Hazard Detection Unit, to catch the load-use case that forwarding cannot resolve because the loaded value is unavailable until the end of the Memory stage. Branch handling was implemented last, since it depends on a stable Execute-stage ALU/comparator output and a working flush mechanism.

3.2 Test-Program (Data Collection) Approach

As this is a hardware-design rather than a data-driven project, “data collection” refers to the construction of targeted test programs — short RV32I instruction sequences, hand-written and assembled to hex, then loaded into instruction memory via \$readmemh — each designed to exercise one category of behaviour: back-to-back arithmetic instructions with register dependencies, load-use sequences, taken and not-taken branches, and combined load/store sequences to the same memory location. Simulation output for each program was compared against the expected architectural state (register file and memory contents) computed manually,



and any mismatch was traced back through the waveform to the responsible module. The complete Verilog source, testbenches, and waveform dump files are maintained in a version-controlled GitHub repository.

3.3 Tools and Technologies

Tool	Role
Verilog HDL	Register-transfer-level description of every processor module
Icarus Verilog	Open-source simulator used to compile and run the RTL and testbench
GTKWave	Waveform viewer used to inspect signal behaviour cycle by cycle
Visual Studio Code	Editor used for writing and organising Verilog source files
Ubuntu Linux	Development and simulation environment

3.4 System and Pipeline Architecture

The processor uses a Harvard-style organisation at the memory interface (separate instruction and data memories) with a single pipelined execution datapath. Instructions are fetched using the PC, decoded and read from the register file in Decode, operated on by the ALU in Execute, used to access data memory in Memory, and written back to the register file in Write Back. Control signals generated by the Decoder in Decode are threaded through the pipeline registers alongside the data they govern. Two cross-cutting blocks sit alongside the linear datapath: the Hazard Detection Unit, comparing the Decode- and Execute-stage instructions to detect load-use hazards, and the Forwarding Unit, comparing the EX/MEM and MEM/WB destination registers against the Execute-stage source registers — both operating combinationally so the pipeline can react within the same cycle.

Stage	Primary Function
Instruction Fetch (IF)	Fetch the instruction addressed by the PC; compute $PC + 4$
Instruction Decode (ID)	Decode instruction, read registers, generate immediate and control signals
Execute (EX)	Perform ALU operation; resolve branch/jump target and outcome
Memory Access (MEM)	Read from or write to data memory for load/store instructions
Write Back (WB)	Write the final result back into the register file



The four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) latch the data and control signals produced by one stage for use by the next, and together define the pipeline's stage boundaries, allowing several instructions to be in flight simultaneously. All four are cleared to a bubble when a flush condition is asserted, and held when a stall condition is asserted. The EX/MEM and MEM/WB registers additionally feed the Forwarding Unit directly, supplying the two forwarding sources used to resolve data hazards without stalling.

3.5 Key Design Modules

Module	Function
Program Counter	Holds and updates the fetch address, subject to stall and branch/jump redirection
Instruction Memory	Combinational, word-addressable array pre-loaded with test-program machine code
Decoder / Control Unit	Generates ALU, memory, register-write, and branch/jump control signals from opcode/funct fields
Register File	32 general-purpose registers (x0–x31); two combinational reads and one synchronous write per cycle; x0 hard-wired to zero
Immediate Generator	Reassembles and sign-extends the immediate field for I, S, B, U, and J instruction formats
ALU	Performs arithmetic, logical, shift, and comparison operations; also computes load/store effective addresses
Branch Adder	Computes branch/jump target address in parallel with the ALU comparison
Data Memory	Byte-addressable array servicing load/store instructions in the Memory stage
Load / Store Units	Handle byte/half-word extraction, sign/zero extension, and correct write-data alignment
Top Module & Testbench	Instantiate and wire all modules; drive clock/reset, load test programs, dump waveforms for GTKWave



3.6 Hazard Handling: Forwarding and Stalling

For an ordinary ALU-to-ALU dependency, the producing instruction's result is available at the end of its Execute stage — exactly when the Forwarding Unit needs it for a dependent instruction now in Execute one cycle later. The Forwarding Unit checks, in priority order, whether the EX/MEM register's destination matches the required source register, then whether MEM/WB's does, and drives ForwardA/ForwardB multiplexers accordingly — giving the more recently produced EX/MEM value priority over the older MEM/WB value. This resolves the common case of read-after-write hazards with no stall cycles.

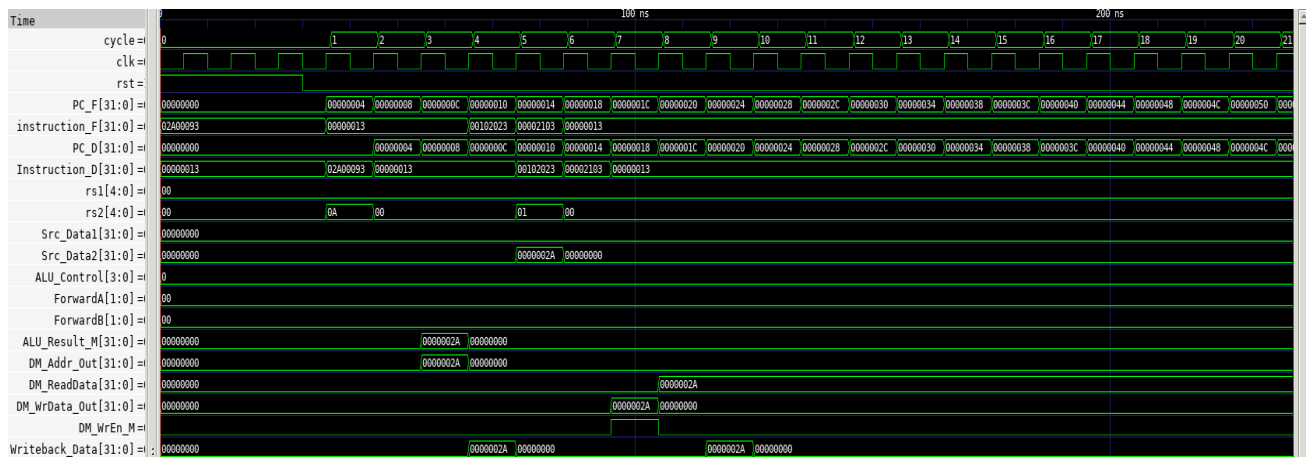


Figure 1: ALU control and forwarding signals traced, confirming correct Execute-stage data forwarding.

A load instruction's value, however, is not available until the end of the Memory stage — one cycle later than an ALU result — so if the very next instruction needs it, forwarding alone cannot deliver it in time. The Hazard Detection Unit checks this condition combinationally (a pending load whose destination register matches a source register of the instruction now in Decode):

When asserted, this de-asserts PC_Write and IF_ID_Write to freeze the PC and the fetched instruction for one cycle, while forcing the ID/EX control fields to zero to insert a bubble into Execute, as summarised below.



On the following cycle the load reaches MEM/WB and its result is forwarded normally, delaying the dependent instruction by exactly the one cycle required and no more.

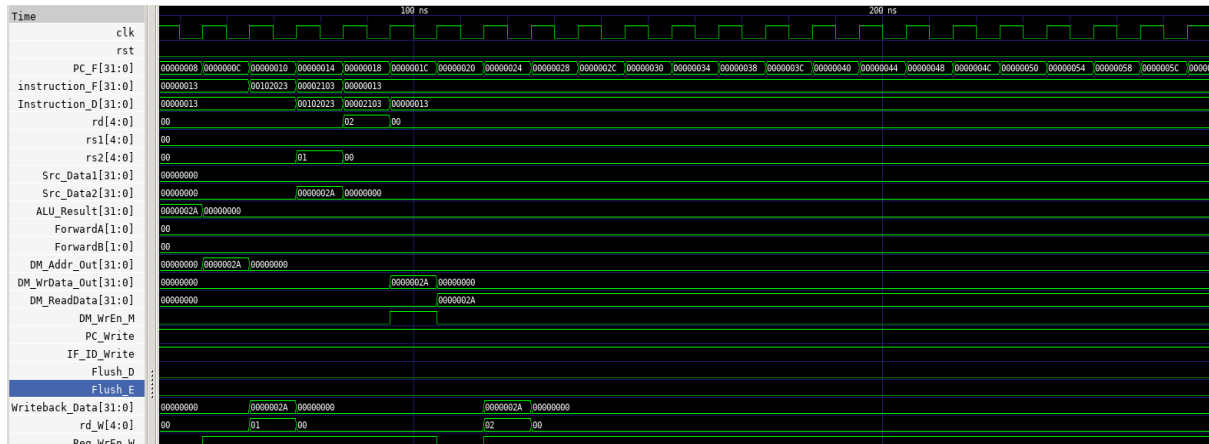


Figure 2: Pipeline stall/flush signals remain inactive, confirming correct control during the ADDI-SW-LW sequence.

3.7 Branch and Jump Handling

All branch and jump outcomes are resolved in the Execute stage. For conditional branches (BEQ, BNE, BLT, BGE, BLTU, BGEU), the ALU/comparator result determines whether the branch is taken, while the Branch Adder computes the target address in parallel. JAL redirects control flow unconditionally and writes PC + 4 as a return address; JALR computes its target from a register plus an immediate and likewise writes PC + 4. Because branches resolve in Execute rather than Decode, a taken branch or jump means the two instructions already fetched into IF/ID and ID/EX are on the wrong path and must be discarded before they can affect architectural state.

Because branches resolve in Execute rather than earlier in the pipeline, the penalty for a taken branch is two cycles — the bubbles inserted by Flush_D and Flush_E. This is a deliberate trade-off: resolving branches later keeps the Decode stage simple, at the cost of a higher misprediction penalty, identified in Section 6 as a candidate for improvement via branch prediction.

3.8 Supported Instruction Set

The processor implements all 37 targeted RV32I instructions, each exercised by at least one hand-written test program during verification.



Category	Instructions Implemented	Count
R-type	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU	10
I-type	ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI	9
Load	LB, LH, LW, LBU, LHU	5
Store	SB, SH, SW	3
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU	6
Jump	JAL, JALR	2
Upper-immediate	LUI, AUIPC	2

3.9 Verification and Results

Functionality was verified through simulation using hand-written RV32I test programs assembled into hex and loaded via \$readmemh. Verification covered all R-type and I-type ALU operations, register-file read/write behavior, load/store variants, branch outcomes with PC redirection and flushing, and cycle-by-cycle pipeline propagation. Data forwarding was validated for dependent ALU instructions without stalls, while load-use hazards were tested with a one-cycle stall followed by correct forwarding. Write-back selection was also verified for ALU results, memory data, and PC + 4. These tests ensured correct operation and enabled failures to be traced to individual pipeline modules.

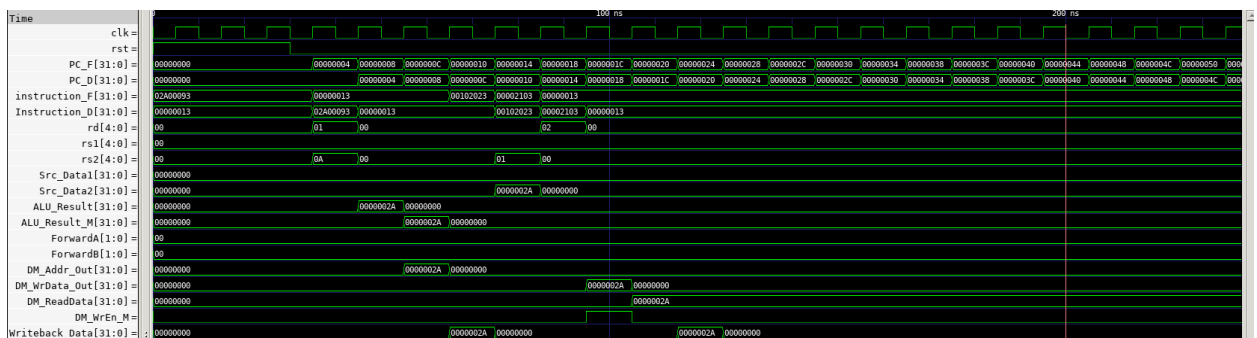
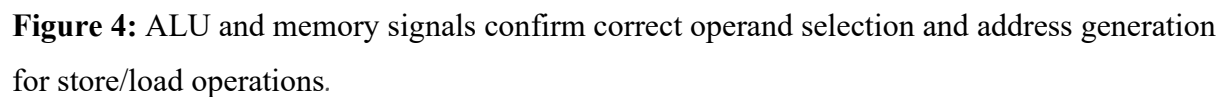


Figure 3: GTKWave trace verifying ADDI → SW → LW execution and correct store-to-load memory consistency (0x2A).



GTKWave was used to trace the VCD file from each run, cycle by cycle, grouping signals for analysis: pipeline instruction words (to track an instruction through IF→ID→EX→MEM→WB); control signals (PC_Write, IF_ID_Write, Control_Zero, Flush_D, Flush_E), to confirm stall/flush conditions assert and de-assert on exactly the expected cycles; forwarding selectors (ForwardA, ForwardB) against the expected source for each dependent instruction; and register-file/data-memory write signals, to confirm correct timing and values. For the load-use hazard specifically, the waveform confirmed the full expected sequence: the stall condition asserts as the load enters EX and the dependent instruction enters ID; PC_Write and IF_ID_Write de-assert that same cycle; Control_Zero inserts a bubble into ID/EX; and on the following cycle, once the load reaches MEM/WB,



ForwardA or ForwardB selects the correct forwarding path and the stall de-asserts. This waveform-level evidence, rather than just a correct final register value, was the basis for confirming that the hazard logic works for the right reasons.

4. Social / Industry Relevance of the Project

RISC-V has grown from an academic instruction set developed at UC Berkeley into an industry-adopted open standard, used today from low-power embedded microcontrollers to data-centre accelerators, precisely because its openness removes the licensing costs and restrictions of proprietary ISAs. For a workforce increasingly involved in semiconductor design and chip-manufacturing initiatives, hands-on RISC-V processor design experience maps directly onto the skill set that the domestic and global semiconductor industry is actively seeking.

For an educated but underemployed rural engineering graduate, a project built entirely with open-source tools and no paid licences demonstrates that industry-relevant computer architecture and digital design skills can be developed without dependency on expensive proprietary EDA tools — directly aligned with SURE Trust's mission of providing hands-on, industry-relevant project experience to educated unemployed rural youth. The skills demonstrated — RTL design, hazard analysis, and simulation-based verification — map directly onto entry-level roles in ASIC design, FPGA development, and digital verification, all active hiring areas within the semiconductor and embedded-systems industry.

5. Learning & Reflection

5.1 Technical Learnings

- A concrete, working-level understanding of how a five-stage pipeline is built in hardware, beyond a textbook block diagram — including the exact timing at which each control signal must assert or de-assert.
- Practical experience distinguishing data hazards resolvable by forwarding from the load-use case that requires a stall, and implementing the detection logic correctly.
- Hands-on familiarity with RV32I instruction encoding, including why immediate fields are split across non-contiguous bit positions across formats, and how to reconstruct them in hardware.



6. Future Scope & Conclusion

6.1 Conclusion

This project successfully achieved its primary goal: the design, implementation, and simulation-based verification of a 32-bit RV32I five-stage pipelined processor in Verilog HDL. All 37 targeted RV32I instructions were implemented and individually verified, the four pipeline registers were correctly integrated to allow multiple instructions in flight simultaneously, and the Forwarding Unit and Hazard Detection Unit were shown, through waveform analysis, to resolve data hazards without stalling in the common case and to correctly insert a single-cycle stall for the load-use case. Branch and jump handling was verified to correctly redirect the program counter and flush incorrectly fetched instructions. The resulting design is a functionally correct, from-first-principles implementation of the architectural technique underlying the vast majority of modern processors, providing both a demonstrable technical deliverable and a solid conceptual foundation for further work in computer architecture and digital design.

6.2 Future Scope

- Branch prediction — replacing the current always-flush approach with a predictor (even a simple static or one-bit dynamic scheme) to reduce the two-cycle penalty on every taken branch.
- Cache memory — introducing instruction and/or data caches for a more realistic memory system and meaningful performance analysis under realistic latency.
- CSR instructions — adding RISC-V Control and Status Registers, a prerequisite for counters, timers, and machine-mode status reporting.
- Interrupt handling — detecting, prioritising, and servicing external and timer interrupts, needed for any real embedded or OS-capable deployment.
- Exceptions — detection and handling of conditions such as illegal instructions and misaligned accesses, with the associated trap/return mechanism.